

Group 6 Coding Project Final Summary – Greedy Dwarf

Shaan Kohli, Brian Rosca, Victoria Rossi, Hristian Tountchev

Project Description & Scope

Greedy Dwarf is an arcade-style, top-down dungeon game where the player navigates through tightly designed environments to collect valuable treasures and avoid a pursuing dragon. At the core of the game's design is a strategic trade-off that borrows from the Knapsack Problem algorithm. The collection of every treasure will add weight, hence slowing down the movement speed of the dwarf and forcing the player to carefully consider the value of items in terms of their ability in navigating fast. Many Points of Interest (POIs), are available with different setups, resources, and challenges along with three playable character classes will make the user want to explore and experiment.

Key Deliverables

The final code includes a rich set of features that meet original design goals and surpass initial concepts:

1. Gameplay Mechanics:

- **Inventory and Weight Management:** Players accumulate treasures of varying weight and value, affecting movement speed and strategy. The system supports adding, dropping, and viewing items with clear visual feedback.
- **Character Classes:** Scout, Miner, and Warrior classes provide distinct playstyles. The Scout moves swiftly but can carry less, the Miner can haul more treasure at the cost of speed, and the Warrior offers a balanced approach.
- **Dynamic Environments and POIs:** Multiple interconnected maps, including special areas like Jungle City, Dragon's Lair, and Crossroads City, encourage exploration. A map state management system ensures seamless transitions and persistent world states.

2. Enemy AI & Pathfinding:

- The chase logic of the dragon uses the A* pathfinding algorithm for effective navigation around obstacles and player tracking. This new level of enemy intelligence greatly raises tension and requires more careful planning, as formerly safe routes may no longer be a reliable means of escape.

3. User Interface & Interaction:

- A clear and responsive UI shows inventory status, score, keys, and load indicators. Common controls and menus, along with tooltips and status messages, improve user experience.

- Leaderboards, user authentication, and profiling enable players to view and compare scores with others, thereby creating replayability and competition.

4. Technical Infrastructure:

- Implemented with React for state management and responsiveness, ensuring smooth performance across platforms.
- Data structures (maps, characters, items) are well-defined and documented. Naming conventions and UML notations were followed to maintain code clarity.

Testing & Quality Assurance

- **Functional Testing:** Verified dwarf movement, dragon pursuit, treasure collection, scoring updates, map transitions, and interactive elements (doors, keys, chests).
- **Character Attribute Tests:** Confirmed each class's distinct speeds and weight thresholds worked as intended.
- **Performance and Reliability:** Frame rate, response times, and platform consistency were validated to achieve stable performance and smooth gameplay.
- **Leaderboard & Search Functionality:** Tested user profiles and score searching to ensure leaderboards update correctly and users can find their records.
- **Regression Testing:** Re-tests after bug fixes were done to make sure the updates didn't introduce new issues. Things like dragon navigation, treasure dropping, and POI transitions now work reliably.

Inspection Results & Improvements

Code inspections resolved a number of issues. Inventory logic changes fixed item handling inaccuracies; the removal of one instance now correctly affects both quantity and total weight. The dragon's A* pathfinding was refined, resolving sporadic obstacle traversal errors. Improved map transition logic kept state consistent between areas. Normalization of UI styling and heavier commenting increased clarity, helping to maintain a consistent look and feel across all interfaces, with overall improved code readability.

Conclusions & Future Directions

The final coding deliverable has a key set of objectives: stable gameplay loops, strategic depth via inventory and character choice, intelligent enemy AI, polished UI. Also, a list of recommendations for the team in regard to implementing continuous integration pipelines, expanding automated testing, maintaining high documentation standards. And then there is the "waiting room" of features that are possible, like POIs, multiplayer modes, more advanced dragon behavior, and story features. In short, Greedy Dwarf is functionally complete, well-tested, and the code is fully documented.